To understand it, you must recall that tensors (in the machine learning sense, where they are just multidimensional arrays) may have 0 or more dimensions that are accessed by corresponding indices. Recall that a scalar (single number) is a 0-tensor, a vector is a 1-tensor, a matrix is a 2-tensor, and so on. The number corresponds to how many indices each tensor has. A vector has one index because each element in a vector can be addressed and accessed by a single nonnegative integer index value. A matrix element is accessed by two indices, its row and column positions. This generalizes to arbitrary dimensions.

If you've made it this far, you're familiar with operations like the inner (dot) product between two vectors and matrix multiplication (either multiplying a matrix with a vector or another matrix). The generalization of these operations to arbitrary order tensors (e.g., the "multiplication" of two 3-tensors) is called a *tensor contraction*. Einstein notation makes it easy to represent and compute any arbitrary tensor contraction, and with self-attention, we're attempting to contract two 3-tensors (and later two 4-tensors) so it becomes necessary to use Einsum or we would have to reshape the 3-tensor into a matrix, do normal matrix multiplication, and then reshape it back into a 3-tensor (which is much less readable than just using Einsum).

This is the general formula for a tensor contraction of two matrices:

$$C_{i,k} = \sum_{j} A_{i,j} B_{j,k}$$

The output on the left, $C_{i,k}$, is the resulting matrix from multiplying matrices A:$i \times j$ and B:$j \times k$ (where i,j,k are the dimensions) such that dimension j for each matrix is the same size (which we know is required to do matrix multiplication). What this tells us is that element $C_{0,0}$, for example, is equal to $\Sigma A_{0,j} B_{j,0}$ for all j. The first element in the output matrix C is computed by taking each element in the first row of A, multiplying it by each element in the first column of B, and then summing these all together. We can figure out each element of C by this process of summing over a particular *shared index* between two tensors. This summation over a